CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Technology and Engineering CSPIT-IT

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical 1: Introduction to React and Component Architecture	
CO/PO	CO1 / PO3, PO5
Objective	To set up a React development environment using Vite and build a static UI using independently structured, reusable components.
Industry Relevance	Mirrors the standard local dev setup used in modern frontend teams Vite-based scaffolding and component reuse are foundational skills expected from day one in any React role.
Prerequisites	• Basic HTML and CSS • Familiarity with JavaScript fundamentals (variables, functions) • Node.js and npm installed on the lab machine
Theory Concepts (Self-Study Reference)	
Concepts to Review	• What React is and why component-based UI development is used over plain HTML/JS • JSX syntax embedding JavaScript expressions inside markup • Functional components and the role of the return statement • Props passing data from a parent component into a child component • Why reusable components reduce duplication and improve maintainability
Architecture / Diagram	

Component Tree	`App.jsx`
	`├── Header.jsx (site title, name)`
	`├── About.jsx (short bio text)`
	`├── Skills.jsx (list of skills, receives props)`
	`└── Footer.jsx (contact / copyright info)`

Problem Definition	
Problem Statement	• Create a React application using Vite for a student portfolio page. • The application must include at least 4 reusable components: Header, About, Skills, and Footer. • Each component must be independently structured and composed into a single-page layout. • No logic or JSX should be duplicated across components. • Props must be used to pass at least one piece of data into a minimum of 2 components.
Key Questions / Analysis	• What is the role of each component in the overall UI structure? • How does React re-render when props change? • Why is component reusability important in large-scale applications?
Supplementary Problems	• Add a NavBar component that highlights the active section. • Pass an array of skills as a prop to the Skills component and render them dynamically. • Add a theme color prop to the Header and apply it as an inline style.
Key Skills Addressed	• Setting up a React project using Vite • Designing and structuring reusable functional components • Passing data between components using props • Composing a single-page layout from multiple components
Learning Outcome	Students will be able to set up a React development environment, create reusable components, and compose them into a working UI using props for data flow.

Lab Session Step-by-Step

Steps

1. Verify Node.js and npm are installed:

```
node -v
npm -v
```

2. Scaffold the React app using Vite:

```
npm create vite@latest student-portfolio -- --template react
cd student-portfolio
npm install
npm run dev
```

3. Inside src/components, create four files: Header.jsx, About.jsx, Skills.jsx, Footer.jsx.
4. Build the Skills component to accept a props array and render it as a list:

```
function Skills({ skillList }) {
  return (
    <ul>
      {skillList.map((s) => <li key={s}>{s}</li>)}
    </ul>
  );
}
```

5. Import and compose all four components inside App.jsx in a single layout.
6. Pass a name prop to Header and a skillList prop to Skills from App.jsx.
7. Run the app and verify all components render correctly without console errors.

Tools / Technology / Resources Required

- Node.js (v18+), npm
- Visual Studio Code
- Vite, React 18+
- Browser DevTools (Console + React DevTools extension)

Total Hours

- Lab Session: 2 hours
- Implementation: 1.5 hours
- Testing / Modification: 0.5 hours
- Total Engagement: 2 hours

Faculty Teaching Guide

Guidance Notes

- Pre-check that lab machines have Node.js installed; keep an offline installer ready as backup.
- Live-demo the Vite scaffold once, slowly, before letting students repeat it independently.
- Walk the room during npm install slow lab Wi-Fi or proxy settings are the most common blocker.
- Stress 'one component, one responsibility' while reviewing student component structure.

Common Mistakes

- Using an outdated Node version, causing the Vite scaffold command to fail.
- Putting all UI code inside App.jsx instead of splitting into separate components.
- Hardcoding values instead of passing them as props.
- Forgetting to save a file before checking the browser, leading to confusion about why changes aren't visible.

Troubleshooting Guide

Symptom	Likely Cause	Fix
npm create vite hangs	Network/proxy blocked	Check lab internet connection; retry on stable network or use offline cache
Port 5173 already in use	Old dev server still running	Stop the previous process or run npm run dev -- --port 5174

Props show as undefined	Prop name mismatch between parent and child	Verify the prop name passed in App.jsx matches the name used in the child component

Student Assignment / Post Laboratory Work

Tasks	• Review the Theory Concepts section and the assigned Coursera module before next week. • Add a Projects component to the portfolio that renders a hardcoded list of 3 projects. • Experiment with passing different prop values and observe how the rendered UI changes.

GitHub Deliverables

Expected Repository Contents	• Public repository named portfolio-<rollno> • Working Vite + React project with 4+ components • Clean .gitignore (node_modules excluded) • At least one meaningful commit message describing the work

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
Environment Setup (3 marks)	All 4 tools (Node, npm, VS Code, Vite) verified via terminal; React app created and runs successfully using Vite without errors.
Project Structure (3 marks)	Components placed in a dedicated /components folder; no unused files; folder structure is clean and logical.
Component Reusability (5 marks)	Minimum 4 components created; each is independently structured; no JSX or logic duplicated across components.
Props Usage (4 marks)	At least 2 components receive and render props correctly; no hardcoded values inside components that should be dynamic.
UI Output (5 marks)	App renders in browser without errors; all 4 components are visible; layout is coherent and components are clearly distinguishable.
Total: 20 marks	**Passing Threshold: 14 / 20 (70%)**

Practical 2: State Management and Routing in React	
CO/PO	CO1 / PO3, PO5
Objective	To implement reactive state management using useState and multi-page navigation using React Router in the existing portfolio application.
Prerequisites	• Practical 1 completed working portfolio app with reusable components • Basic understanding of JavaScript array/object handling
Theory Concepts (Self-Study Reference)	
Coursera Pointer	See Week 2 IBM: Developing Front-End Apps with React, Module 2 (props, event handling, component composition, useState hook).
Architecture / Diagram	

Diagram	`App.jsx (wrapped in <BrowserRouter>)`
	`  ├── NavBar.jsx  (links to all routes)`
	`  ├── Route: "/"  → Home.jsx`
	`  ├── Route: "/projects" → Projects.jsx`
	`  └── Route: "/contact"  → Contact.jsx (controlled form, useState)`

Problem Definition	
Problem Statement	• Extend the portfolio application from Practical 1 by adding React Router. • Implement a navigation bar with at least 3 routes: Home, Projects, and Contact. • Each route must render a distinct component without a full page reload. • Add at least 2 useState variables used meaningfully one for toggling UI visibility and one for managing a form input on the Contact page. • The Contact page must include a controlled input that captures and displays user input in real time.
Key Questions / Analysis	• What is the role of each component in the overall UI structure? • How does React re-render when props change? • Why is component reusability important in large-scale applications?
Supplementary Problems	• Add a 404 Not Found route that renders a custom error component for unknown paths. • Implement a dark/light mode toggle using useState that applies a CSS class to the root element. • Store the contact form input in state and display a live character count below the input.
Key Skills Addressed	• Installing and configuring React Router v6 • Defining and navigating between routes without page reload • Managing component state using the useState hook • Building controlled form inputs tied to state
Learning Outcome	Students will be able to implement client-side routing with React Router and manage dynamic UI state using useState, resulting in a multi-page application without full page reloads.
Lab Session Step-by-Step	

Steps	1. Install React Router in the existing portfolio project:
	`npm install react-router-dom`
	2. Wrap the App component with BrowserRouter in main.jsx:
	`import { BrowserRouter } from 'react-router-dom';`
	`<BrowserRouter>`
	`  <App />`
	`</BrowserRouter>`
	3. Define 3 routes inside App.jsx using Routes and Route:
	`<Routes>`

	`<Route path="/" element={<Home />} />`
	`<Route path="/projects" element={<Projects />} />`
	`<Route path="/contact" element={<Contact />} />`
	`</Routes>`
	4. Build the NavBar using Link (not <a> tags) to avoid full page reloads.
	5. Inside Contact.jsx, create a controlled input tied to useState:
	`const [message, setMessage] = useState('');`
	`<input value={message} onChange={(e) =>`
	`setMessage(e.target.value)} />`
	6. Add a second useState variable to toggle visibility of an element (e.g., a help tooltip).
	7. Test navigation across all 3 routes and confirm no page reload occurs.

Tools / Technology / Resources Required	• Node.js (v18+), npm • Visual Studio Code • React 18+, React Router v6 • Browser DevTools
Total Hours	• Lab Session: 2 hours • Implementation: 1.5 hours • Testing / Modification: 0.5 hours • Total Engagement: 2 hours
Faculty Teaching Guide	
Guidance Notes	• Emphasize Link vs <a> this is the most common point of confusion for students new to SPA routing. • Live-demo useState updating the UI in real time before students attempt the Contact form. • Check that BrowserRouter wraps the App only once, typically in main.jsx, not repeated elsewhere. • Walk through how routes map to components before students start writing code.
Common Mistakes	• Using <a href="..."> instead of <Link to="...">, causing full page reloads. • Forgetting to wrap the app in BrowserRouter, resulting in a 'useRoutes() may be used only in the context of a Router' error. • Using props instead of state for values that should change dynamically (like form input). • Not providing a unique path for each Route, causing routes to render incorrectly.

Troubleshooting Guide

Symptom	Likely Cause	Fix
Blank page after adding routes	BrowserRouter not wrapping App component	Confirm BrowserRouter wraps <App /> in main.jsx
Clicking nav link reloads page	Used <a> tag instead of <Link>	Replace all navigation anchors with <Link to="..."> from react-router-dom
Form input not updating on typing	Missing onChange handler or value not tied to state	Ensure input has both value={state} and onChange={set state}
Route shows wrong component	Path mismatch or routes defined in wrong order	Double check path strings match exactly; order routes from specific to general

Student Assignment / Post Laboratory Work

Tasks	• Review the Theory Concepts section and the assigned Coursera module before next week. • Add a 404 Not Found route for any undefined path. • Implement a dark/light mode toggle using a second useState variable.

GitHub Deliverables	
Expected Repository Contents	• Same repository as Practical 1, updated with new commits • Working multi-route app with NavBar, Home, Projects, Contact • At least one commit specifically for routing implementation • Updated README noting new routes added

Rubrics (Practical Evaluation / Viva)	
Criteria (Marks)	**Description**
React Router Setup (4 marks)	React Router v6 installed; BrowserRouter wraps the app correctly; no console routing errors on load.
Route Configuration (4 marks)	Minimum 3 routes defined; each route renders the correct component; navigation links work without triggering a page reload.
useState UI Toggle (4 marks)	At least one UI element toggles visibility or appearance based on a state variable; state change is reflected immediately in the UI.
Controlled Form Input (5 marks)	Contact page form input is a controlled component tied to state via value and onChange; input value updates correctly on every keystroke.
Navigation UX (3 marks)	Active route is visually distinguishable in the NavBar; navigating between all 3 routes works correctly without errors.
Total: 20 marks	**Passing Threshold: 14 / 20 (70%)**

API Integration and Data Rendering in React	
CO/PO	CO1 / PO3, PO5
Objective	To consume a REST API in React and handle asynchronous data with loading and error states.
Prerequisites	• Practical 1 and 2 completed multi-route portfolio app with state management • Basic understanding of JavaScript Promises and async/await
Theory Concepts (Self-Study Reference)	
Coursera Pointer	See Week 3 IBM: Developing Front-End Apps with React, Module 3 (React hooks, form management, connecting to external servers, async data fetching).
Architecture / Diagram	

Diagram	`Projects.jsx`
	`  ├── useEffect()  → triggers fetch on mount`
	`  ├── useState: data, loading, error`
	`  ├── [loading] → <Spinner />`
	`  ├── [error]   → <ErrorMessage />`
	`  └── [success] → <RepoList data={data} />`

Problem Definition	
Problem Statement	• Integrate a public REST API (e.g., GitHub API) into the portfolio application to fetch and display a list of repositories dynamically. • Use the Projects page built in Practical 2 as the integration point. • Implement a loading spinner component shown while the request is in progress. • Implement an error message component shown if the API call fails. • Render at least the repository name and URL for each item returned.
Key Questions / Analysis	• Why is useEffect required to trigger a fetch on component mount instead of calling fetch directly in the component body? • What is the difference between a loading state and an error state, and why must both be handled separately? • How would the user experience change if loading and error states were not implemented?
Supplementary Problems	• Add a retry button that re-triggers the fetch when an error occurs. • Add a search input that filters the rendered repository list by name. • Display the repository's star count alongside its name.
Key Skills Addressed	• Fetching data from a REST API using fetch or axios • Using useEffect to trigger side effects on component mount • Managing asynchronous state loading, success, and error • Conditionally rendering UI based on request state
Learning Outcome	Students will be able to consume a REST API in React, manage asynchronous state correctly, and provide appropriate UI feedback during all stages of a network request.
Lab Session Step-by-Step	

Steps	1. Inside Projects.jsx, set up 3 state variables for data, loading, and error:
	`const [repos, setRepos] = useState([]);`
	`const [loading, setLoading] = useState(true);`
	`const [error, setError] = useState(null);`
	2. Use useEffect to fetch data when the component mounts:
	`useEffect(() => {`
	`  fetch('https://api.github.com/users/<username>/repos')`
	`  .then((res) => res.json())`
	`  .then((data) => setRepos(data))`

	`.catch((err) => setError(err.message))`
	`.finally(() => setLoading(false));`
	`}, []);`
	3. Conditionally render based on state:
	`if (loading) return <Spinner />;`
	`if (error) return <ErrorMessage message={error} />;`
	4. Map over the repos array and render name and html_url for each repository.
	5. Test the happy path confirm data renders correctly after loading.
	6. Test the error path temporarily break the API URL and confirm the error message displays.

Tools / Technology / Resources Required	• Node.js (v18+), npm • Visual Studio Code • React 18+, Fetch API or Axios • Browser DevTools (Network tab)
Total Hours	• Lab Session: 2 hours • Implementation: 1.5 hours • Testing / Modification: 0.5 hours • Total Engagement: 2 hours
Faculty Teaching Guide	
Guidance Notes	• Use a public API requiring no authentication (GitHub API is ideal) to avoid API key issues in lab. • Demonstrate breaking the API URL deliberately to show the error state students rarely test this on their own. • Explain why the dependency array [] in useEffect matters fetching on every render is a very common bug to anticipate. • Walk through the difference between .then() chains and async/await if students choose to use the latter.
Common Mistakes	• Calling fetch directly inside the component body without useEffect, causing infinite re-fetching. • Omitting the dependency array in useEffect, causing the request to fire on every render. • Not handling the error case at all, causing the app to crash or show a blank screen on failure. • Forgetting to set loading back to false after the fetch completes, leaving the spinner stuck.
Troubleshooting Guide	

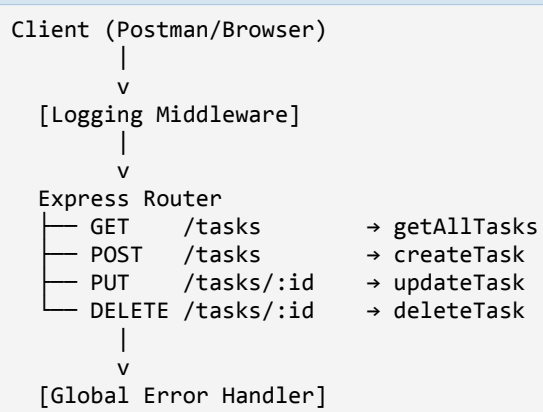
Symptom	Likely Cause	Fix
Infinite network requests in console	Missing dependency array in useEffect	Add an empty array [] as the second argument to useEffect
Spinner never disappears	setLoading(false) not called in .finally() or after both success/error paths	Use .finally() to guarantee loading is set to false regardless of outcome
App crashes on API failure	No error boundary or conditional check before rendering data	Add error state handling and conditionally render ErrorMessage before the data list
CORS error in console	API does not allow cross-origin requests from localhost	Use a CORS-friendly public API like GitHub's; do not attempt to bypass CORS with browser flags

Student Assignment / Post Laboratory Work	
Tasks	• Review the Theory Concepts section and the assigned Coursera module before next week. • Add a retry button that re-triggers the fetch when an error occurs. • Add a simple search/filter input above the repository list.

GitHub Deliverables	
Expected Repository Contents	• Same repository as Practical 1 and 2, updated with new commits • Working API integration on the Projects page with loading and error states • At least one commit specifically for API integration • Updated README noting the API used and any setup steps

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
API Call Implementation (5 marks)	fetch or axios used correctly; API URL is valid; request fires on component mount using useEffect.
Data Rendering (5 marks)	API response parsed and mapped correctly; list of repositories displayed with at least name and URL.
Loading State (4 marks)	Loading spinner/indicator visible while data is being fetched; disappears after data loads.
Error Handling (4 marks)	Error message component renders on failed API call; does not crash the app.
Code Cleanliness (2 marks)	No console.log left in code; async logic separated from JSX.
Total: 20 marks	**Passing Threshold: 14 / 20 (70%)**

Practical 4: Building a RESTful API with Node.js and Express	
CO/PO	CO2 / PO3, PO5
Objective	To design and implement a RESTful backend server with complete CRUD endpoints using Express middleware pipeline.
Prerequisites	• Basic JavaScript (functions, objects, arrays, async basics) • Understanding of HTTP methods (GET, POST, PUT, DELETE) and status codes
Theory Concepts (Self-Study Reference)	
Coursera Pointer	See Week 4 IBM: Node.js & MongoDB Developing Back-end Database Applications, Module 1 (RESTful API design with Node.js and Express, middleware pipeline, request lifecycle).
Architecture / Diagram	
Diagram	
Problem Definition	
Problem Statement	• Build a Node/Express backend for a Task Management system. • Implement REST endpoints for creating, reading, updating, and deleting tasks (in-memory array, no database yet). • Apply a request logging middleware that logs method, URL, and timestamp for every incoming request. • Apply a global error handling middleware as the last middleware in the pipeline. • Use correct HTTP status codes (200, 201, 404, 500) for each response.
Key Questions / Analysis	• Why must the error handling middleware be defined last in the middleware chain? • What is the difference between app.use() and a route-specific middleware? • Why is it considered bad practice to send raw error stack traces to the client?
Supplementary Problems	• Add a middleware that rejects requests without a Content-Type: application/json header on POST/PUT. • Add a route-specific middleware that validates the task ID format before reaching the controller. • Add a 404 handler for undefined routes that returns a structured JSON response.
Key Skills Addressed	• Setting up an Express server and defining RESTful routes • Writing and applying custom middleware functions • Structuring a request-response-error pipeline correctly • Returning appropriate HTTP status codes for each operation
Learning Outcome	Students will be able to design and implement a complete RESTful API using Express, including a working middleware pipeline with logging and

	centralized error handling.
Lab Session Step-by-Step	

Steps	1. Initialize a new Node project and install Express:
	`mkdir task-manager-api && cd task-manager-api`
	`npm init -y`
	`npm install express`
	2. Create server.js and set up a basic Express app:
	`const express = require('express');`
	`const app = express();`
	`app.use(express.json());`
	`app.listen(5000, () => console.log('Server running on port 5000'));`
	3. Add a request logging middleware applied globally:
	`app.use((req, res, next) => {`
	`  console.log(`${req.method} ${req.url} - ${new Date().toISOString()}`);`
	`  next();`
	`});`
	4. Implement the 4 CRUD routes using an in-memory array as temporary storage.
	5. Add a global error handling middleware as the final middleware:
	`app.use((err, req, res, next) => {`
	`  console.error(err.stack);`
	`  res.status(500).json({ error: 'Something went wrong' });`
	`});`
	6. Test all 4 endpoints using Postman or Thunder Client, verifying correct status codes.

Tools / Technology / Resources Required	• Node.js (v18+), npm • Express.js • Postman or Thunder Client • Visual Studio Code
Total Hours	• Lab Session: 2 hours • Implementation: 1.5 hours • Testing / Modification: 0.5 hours • Total Engagement: 2 hours
Faculty Teaching Guide	
Guidance Notes	• Demonstrate the request lifecycle on the whiteboard/diagram before coding students often treat middleware as 'magic'. • Emphasize calling next() in every middleware that doesn't end the response, or requests will hang indefinitely. • Show a live example of the error handler catching a deliberately thrown error. • Walk through Postman setup once as a class before students work independently.
Common Mistakes	• Forgetting to call next() inside custom middleware, causing the request to hang. • Placing the error handling middleware before the routes instead of after all of them. • Not setting app.use(express.json()), causing req.body to be undefined on POST/PUT. • Returning inconsistent response formats across different endpoints.

Troubleshooting Guide

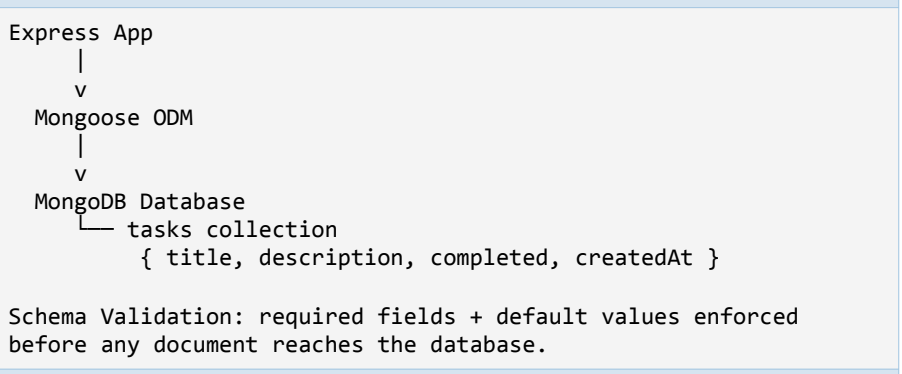
Symptom	Likely Cause	Fix
Request hangs indefinitely	Middleware does not call next() or send a response	Ensure every middleware either calls next() or ends the response with res.send/json
req.body is undefined	Missing express.json()	Add app.use(express.json())

Postman shows 'Cannot GET /tasks'	Route path mismatch or server not restarted after changes	Verify route path spelling and restart the Node server

Student Assignment / Post Laboratory Work	
Tasks	• Review the Theory Concepts section and the assigned Coursera module before next week. • Add a middleware that rejects POST/PUT requests missing a Content-Type header. • Add a custom 404 handler for undefined routes.
GitHub Deliverables	
Expected Repository Contents	• New repository named task-manager-api-<rollno> • Working Express server with all 4 CRUD routes and middleware pipeline • Clean .gitignore (node_modules excluded) • At least one meaningful commit message describing the work

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
Server Setup (3 marks)	Express server runs on defined port; confirmed via terminal and browser/Postman.
REST Endpoints (8 marks)	All 4 CRUD endpoints (GET, POST, PUT, DELETE) implemented; correct HTTP methods and status codes used.
Request Logging Middleware (4 marks)	Custom middleware logs method, URL, and timestamp for every request; applied globally.
Global Error Handling (5 marks)	Error handling middleware defined last in pipeline; catches unhandled errors; returns structured JSON error response.
Total: 20 marks	**Passing Threshold: 14 / 20 (70%)**

Practical 5: MongoDB Integration and Schema Design with Mongoose	
CO/PO	CO2, CO3 / PO3, PO5
Objective	To connect a MongoDB database to an Express server and enforce data validation through Mongoose schema.
Prerequisites	• Practical 4 completed working Express server with in-memory CRUD • Basic understanding of NoSQL/document-based data (vs relational tables)
Theory Concepts (Self-Study Reference)	
Coursera Pointer	See Week 5 IBM: Node.js & MongoDB Developing Back-end Database Applications, Module 2 (data and databases intro, MongoDB CRUD operations, Mongoose schema design and validation).
Architecture / Diagram	
Diagram	
Problem Definition	
Problem Statement	• Extend the Task Management backend from Practical 4 by connecting MongoDB using Mongoose. • Define a Task schema with at least 4 fields: title (String, required), description (String), completed (Boolean, default false), createdAt (Date, default Date.now). • Replace the in-memory array from Practical 4 with real Mongoose model operations. • Test all CRUD operations against the live database using Postman. • Ensure validation errors are returned as structured JSON, not raw Mongoose error objects.
Key Questions / Analysis	• What is the purpose of a schema in a NoSQL database like MongoDB, given that MongoDB itself is schema-less? • Why is it important to define required fields and default values at the schema level rather than relying on frontend validation alone? • What happens internally when a document fails Mongoose validation where is the request stopped?
Supplementary Problems	• Add a priority field to the schema restricted to an enum of ['low', 'medium', 'high']. • Add a pre-save hook that automatically trims whitespace from the title field. • Implement a GET /tasks/:id endpoint that returns a 404 JSON response if the ID does not exist.
Key Skills Addressed	• Connecting a Node/Express app to MongoDB using Mongoose • Designing a schema with field types, required constraints, and defaults • Performing CRUD operations using Mongoose model methods • Returning structured validation error responses
Learning Outcome	Students will be able to connect an Express application to MongoDB using Mongoose, define a validated schema, and perform real CRUD operations against a live database.

Lab Session Step-by-Step

Steps

1. Install Mongoose and create a .env file for the connection string:

```
npm install mongoose dotenv
```

2. Connect to MongoDB inside server.js:

```
const mongoose = require('mongoose');
require('dotenv').config();
mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('MongoDB connected'))
  .catch((err) => console.error(err));
```

3. Define the Task schema and model in a separate models/Task.js file:

```
const taskSchema = new mongoose.Schema({
  title: { type: String, required: true },
  description: { type: String },
  completed: { type: Boolean, default: false },
  createdAt: { type: Date, default: Date.now }
});
module.exports = mongoose.model('Task', taskSchema);
```

4. Replace the in-memory array logic from Practical 4 with Mongoose methods: Task.find(), Task.create(), Task.findByIdAndUpdate(), Task.findByIdAndDelete().
5. Wrap each route in a try/catch and pass errors to the global error handler using next(err).
6. Test all 4 endpoints in Postman; confirm data persists in MongoDB after server restart.

Tools / Technology / Resources Required

- Node.js (v18+), npm
- Express.js, Mongoose
- MongoDB (local or Atlas)
- Postman or Thunder Client

Total Hours

- Lab Session: 2 hours
- Implementation: 1.5 hours
- Testing / Modification: 0.5 hours
- Total Engagement: 2 hours

Faculty Teaching Guide

Guidance Notes

- Decide in advance: local MongoDB install vs MongoDB Atlas free tier demonstrate connection string setup for whichever is chosen.
- Show the MongoDB Compass GUI (or Atlas web UI) so students can visually confirm documents are being created.
- Emphasize that schema validation happens at the Mongoose layer, not in the database itself important for understanding NoSQL flexibility vs application-level enforcement.
- Walk through one validation failure live so students see the actual error shape before they handle it.

Common Mistakes

- Hardcoding the MongoDB connection string directly in code instead of using a .env file.
- Forgetting required: true on fields that should never be empty, allowing invalid documents to be saved.
- Not wrapping database calls in try/catch, causing unhandled promise rejections to crash the server.
- Sending the raw Mongoose ValidationError object to the client instead of a clean, structured message.

Troubleshooting Guide

Symptom	Likely Cause	Fix
MongoDB connection times out	Incorrect connection string or IP not whitelisted (Atlas)	Verify MONGO_URI in .env; if using Atlas, whitelist the lab's IP address or allow access from anywhere for lab use
Validation error not caught	Mongoose operation not	Wrap all async Mongoose calls in

Cannot find module 'dotenv'	Package not installed	Run npm install dotenv and ensure require('dotenv').config() is called at the top of server.js

Student Assignment / Post Laboratory Work	
Tasks	• Review the Theory Concepts section and the assigned Coursera module before next week. • Add a priority field to the schema restricted to an enum of low/medium/high. • Implement a GET /tasks/:id endpoint with proper 404 handling.
GitHub Deliverables	
Expected Repository Contents	• Same repository as Practical 4, updated with new commits • Working MongoDB-backed CRUD API with Mongoose schema and validation • .env excluded via .gitignore; .env.example provided instead • At least one commit specifically for MongoDB/Mongoose integration

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
MongoDB Connection (3 marks)	Mongoose connects to MongoDB successfully; connection string in .env file; connection status logged.
Schema Design (5 marks)	Task schema has minimum 4 fields; correct data types assigned; at least 2 required constraints and 1 default value defined.
CRUD via Mongoose (8 marks)	All 4 operations tested and working against live MongoDB; correct Mongoose methods used (save, find, findByIdAndUpdate, findByIdAndDelete).
Validation Enforcement (4 marks)	Missing required fields return validation error; error message is descriptive and returned as JSON.
Total: 20 marks	**Passing Threshold: 14 / 20 (70%)**

Practical 6: Full Stack Integration React + Node + MongoDB	
CO/PO	CO1, CO2 / PO3, PO5
Objective	To wire the React frontend to the Node/Express/MongoDB backend into a fully functional full-stack application.
Prerequisites	• Practicals 1–3 completed working React UI with routing and API consumption patterns • Practicals 4–5 completed working Node/Express/MongoDB backend with CRUD
Theory Concepts (Self-Study Reference)	
Coursera Pointer	See Week 6 IBM: Node.js & MongoDB Developing Back-end Database Applications, Module 3 (connecting frontend to backend, end-to-end API calls, full-stack flow).
Architecture / Diagram	

Diagram	`React Frontend (localhost:5173)`
	`  |  fetch/axios calls`
	`  v`
	`Express Backend (localhost:5000)`
	`  |  Mongoose`
	`  v`
	`  MongoDB Database`
	
	`Flow: UI Form → POST /tasks → MongoDB → UI re-fetches → updated list rendered`

Problem Definition	
Problem Statement	• Connect the React Task UI to the Node+MongoDB backend built in Practicals 4 and 5. • Replace the GitHub repo data from Practical 3 with task data from your own backend. • The application must support creating, viewing, updating, and deleting tasks from the React interface through API calls. • All data must be persisted in MongoDB confirmed by refreshing the browser and seeing data remain. • Handle loading and error states for every API interaction, not just the initial fetch.
Key Questions / Analysis	• What changes are required on the backend (CORS) to allow the React dev server to call the Express API? • Why should the UI re-fetch or update local state after a successful POST/PUT/DELETE rather than assuming success silently? • What is the risk of not handling errors on write operations (POST/PUT/DELETE) the same way as read operations (GET)?
Supplementary Problems	• Add an optimistic UI update for task creation show the new task in the list immediately, before the server confirms. • Add a confirmation dialog before deleting a task. • Add a toast/notification component that shows success or failure after each operation.
Key Skills Addressed	• Configuring CORS on an Express server for local frontend-backend communication • Calling backend CRUD endpoints from React using fetch or axios • Synchronizing UI state with server state after write operations • Handling loading and error states across multiple API interactions in one component
Learning Outcome	Students will be able to integrate a React frontend with a Node/Express/MongoDB backend into a complete, working full-stack application with proper state synchronization.

Lab Session Step-by-Step

Steps	1. Install and configure CORS on the Express backend:
	`npm install cors`
	`const cors = require('cors');`
	`app.use(cors());`
	2. In React, create a central api.js file with the base backend URL:
	`const BASE_URL = 'http://localhost:5000';`
	`export const getTasks = () => fetch(`${BASE_URL}/tasks`).then(res => res.json());`
	3. Replace the Practical 3 GitHub fetch logic with calls to your own /tasks endpoint.
	4. Build a task creation form that POSTs to /tasks and updates local state on success.
	5. Implement update (PUT) and delete (DELETE) actions, each followed by a state update or re-fetch.
	6. Wrap each API call in proper loading/error handling, reusing the pattern from Practical 3.
	7. Test the complete flow: create → view → update → delete, refreshing the browser to confirm persistence.

Tools / Technology / Resources Required	• Node.js (v18+), npm • React 18+, Express.js, Mongoose • cors package • Postman (for backend verification), Browser DevTools
Total Hours	• Lab Session: 2 hours • Implementation: 1.5 hours • Testing / Modification: 0.5 hours • Total Engagement: 2 hours

Faculty Teaching Guide

Guidance Notes	• This is the first practical where two dev servers (React on 5173, Express on 5000) must run simultaneously demonstrate running both in separate terminals before students attempt it. • CORS errors are the single most common blocker in this session show what a CORS error looks like in the browser console before students hit it themselves. • Stress that frontend state should be updated based on the backend response, not assumed this is the core integration concept being tested. • Reserve the last 15 minutes for a full end-to-end demo check: create, edit, delete, and refresh.
Common Mistakes	• Forgetting to start the backend server, then debugging the frontend for an unrelated 'fetch failed' error. • Not enabling CORS, resulting in browser console errors blocking all requests. • Updating local UI state optimistically without confirming the backend write actually succeeded. • Hardcoding the backend URL inconsistently across multiple files instead of using one shared base URL constant.

Troubleshooting Guide

Symptom	Likely Cause	Fix
CORS error in browser console	cors middleware not enabled on Express server	Run npm install cors and add app.use(cors()) before route definitions
fetch failed / Network Error	Backend server not running or wrong port	Confirm Express server is running on port 5000 in a separate terminal
UI shows stale data after edit/delete	Local state not updated after successful API call	Re-fetch the task list or manually update state array after each write operation

Data disappears after refresh	Still reading from local component state instead of backend	Confirm GET /tasks is called on mount and populates state from the database, not hardcoded values

Student Assignment / Post Laboratory Work	
Tasks	• Review the Theory Concepts section and the assigned Coursera module before next week. • Add a confirmation dialog before deleting a task. • Add a simple toast notification for successful create/update/delete actions.

GitHub Deliverables	
Expected Repository Contents	• Two repositories (or one monorepo) clearly separating frontend and backend • Working end-to-end flow: create, read, update, delete tasks from the UI • README with instructions to run both frontend and backend locally • At least one commit specifically for full-stack integration

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
API Connection from React (5 marks)	React correctly calls backend endpoints; base URL configured; no CORS errors in console.
Create Operation (4 marks)	New task submitted from React form; persisted in MongoDB; appears in UI without page refresh.
Read Operation (3 marks)	Task list fetched from backend on load; renders correctly in React.
Update Operation (4 marks)	Edit functionality updates task via API; MongoDB record changes reflected in UI.
Delete Operation (4 marks)	Delete button removes task via API; UI updates immediately without full reload.
Total: 20 marks	**Passing Threshold: 14 / 20 (70%)**

Practical 7: Authentication and Middleware Pipeline	
CO/PO	CO2, CO6 / PO3, PO5
Objective	To implement JWT-based authentication and input validation as part of the Express middleware pipeline.
Prerequisites	• Practical 6 completed working full-stack Task Management application • Basic understanding of hashing and tokens (conceptual, covered in self-study material)
Theory Concepts (Self-Study Reference)	
Coursera Pointer	See Week 7 IBM: Node.js & MongoDB Developing Back-end Database Applications, Module 4 (authentication and authorization, JWT token generation, error handling middleware, request validation and sanitization).
Architecture / Diagram	

Diagram	`POST /register → hash password → save User → 201`
	`POST /login → verify password → sign JWT → return token`
	
	`Protected Route Flow:`
	`Client Request (Authorization: Bearer <token>)`
	`↓`
	`[Auth Middleware] → verifies token → req.user set`
	`↓`
	`[Validation Middleware] → checks input`
	`↓`
	`Controller (task routes)`

Problem Definition	
Problem Statement	• Add user registration and login functionality to the Task Management application. • Hash passwords using bcrypt before saving the User document to MongoDB. • Implement JWT token generation on successful login, with a reasonable expiry (e.g., 1 hour). • Protect all task routes using an authentication middleware that verifies the JWT. • Add server-side input validation that rejects malformed requests (e.g., missing title) before they reach the database.
Key Questions / Analysis	• Why must passwords be hashed before storage instead of saved as plain text, even in a lab/demo project? • What does the authentication middleware actually verify, and what happens if the token is missing or expired? • Why should input validation happen on the server even if the frontend already validates the same fields?
Supplementary Problems	• Add a /me endpoint that returns the currently logged-in user's details using the decoded JWT. • Implement token expiry handling on the frontend redirect to login if a protected request returns 401. • Add a logout mechanism that clears the token from the client.
Key Skills Addressed	• Hashing passwords securely using bcrypt • Generating and verifying JWT tokens • Writing authentication middleware to protect routes • Implementing server-side input validation and sanitization
Learning Outcome	Students will be able to implement a complete authentication flow using

	JWT, protect backend routes with middleware, and enforce server-side input validation.
Lab Session Step-by-Step	

Steps	1. Install required packages:
	`npm install bcryptjs jsonwebtoken`
	2. Create a User schema with email and hashed password fields.
	3. Implement the register route hash the password before saving:
	`const hashedPassword = await bcrypt.hash(password, 10);`
	`const user = await User.create({ email, password: hashedPassword });`
	4. Implement the login route compare password and sign a JWT:
	`const isMatch = await bcrypt.compare(password, user.password);`
	`const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '1h' });`
	5. Create an auth middleware that verifies the token from the Authorization header:
	`const token = req.headers.authorization?.split(' ')[1];`
	`const decoded = jwt.verify(token, process.env.JWT_SECRET);`
	`req.user = decoded;`
	6. Apply the auth middleware to all task routes.
	7. Add a validation middleware that checks required fields exist before reaching the controller.
	8. Test the full flow in Postman: register → login → copy token → access protected task routes.

Tools / Technology / Resources Required	• Node.js (v18+), npm • bcryptjs, jsonwebtoken • Express.js, Mongoose • Postman (for token-based testing)
Total Hours	• Lab Session: 2 hours • Implementation: 1.5 hours • Testing / Modification: 0.5 hours • Total Engagement: 2 hours
Faculty Teaching Guide	
Guidance Notes	• Walk through why bcrypt and not plain SHA hashing is used bcrypt is intentionally slow and salts automatically, which matters for security. • Demonstrate testing protected routes in Postman using the Authorization header with Bearer <token> before students attempt the frontend. • Emphasize JWT_SECRET must come from .env, never hardcoded connect this back to the secure storage principle in the syllabus. • Reserve time to show what happens with an expired or tampered token this builds real understanding of why verification matters.
Common Mistakes	• Storing passwords as plain text or using a reversible encryption instead of bcrypt hashing. • Forgetting to send the Authorization header when testing protected routes, then assuming the middleware is broken. • Hardcoding the JWT secret directly in code instead of using an environment variable. • Not handling the case where jwt.verify() throws on an invalid/expired token, crashing the server.
Troubleshooting Guide	

Symptom	Likely Cause	Fix
401 Unauthorized on every request	Authorization header missing or malformed in Postman/client	Set header as Authorization: Bearer <token> exactly, including the space after Bearer
Server crashes on invalid token	jwt.verify() not wrapped in	Wrap token verification in try/catch

JWT_SECRET is undefined	.env file not loaded or variable name mismatch	Confirm dotenv.config() runs before JWT_SECRET is used and the .env key name matches exactly

Student Assignment / Post Laboratory Work	
Tasks	• Review the Theory Concepts section and the assigned Coursera module before next week. • Add a /me endpoint that returns the logged-in user's details from the decoded token. • Implement basic frontend handling for 401 responses (redirect to a login prompt).

GitHub Deliverables	
Expected Repository Contents	• Same repository as Practical 6, updated with new commits • Working register/login flow with JWT-protected task routes • .env excluded via .gitignore; JWT_SECRET never committed • At least one commit specifically for authentication implementation

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
User Registration (3 marks)	Registration endpoint accepts name, email, password; password hashed before saving to MongoDB.
JWT Login Flow (5 marks)	Login endpoint validates credentials; returns signed JWT token with expiry; tested via Postman.
Auth Middleware (5 marks)	Protected routes reject requests without valid token; return 401; valid token grants access.
Input Validation (4 marks)	At least 3 validation rules enforced (e.g., email format, password length, required fields); invalid input returns descriptive error.
Error Response Structure (3 marks)	All errors returned as consistent JSON with status code and message; no stack traces exposed.
Total: 20 marks	**Passing Threshold: 14 / 20 (70%)**

Practical 8: Performance Optimization and Lazy Loading in React	
CO/PO	CO1 / PO3, PO5
Objective	To improve frontend performance using lazy loading and code splitting techniques.
Prerequisites	• Practicals 1–6 completed multi-route React app with full-stack integration • Basic understanding of how the browser loads and parses JavaScript bundles
Theory Concepts (Self-Study Reference)	
Coursera Pointer	See Week 8 IBM: Developing Front-End Apps with React, Module 4 (advanced React features, hooks for optimization). Supplement with browser DevTools Performance/Network tab profiling.
Architecture / Diagram	

Diagram	`Before (single bundle):`
	`  main.bundle.js  ──────────────►  [Home][Projects][Contact] all loaded upfront`
	
	`After (code-split):`
	`  main.bundle.js  ──► loads on first visit`
	`  Home.chunk.js  ──► loaded only when /  is visited`
	`  Projects.chunk.js ──► loaded only when /projects is visited`
	`  Contact.chunk.js ──► loaded only when /contact is visited`

Problem Definition	
Problem Statement	• Optimize the React frontend of the Task Management application by implementing lazy loading for route-based components. • Apply React.lazy() and Suspense to at least the Projects and Contact routes (or equivalent route-based components). • Provide a meaningful fallback UI while a lazy-loaded chunk is loading. • Measure and compare bundle size and load time before and after optimization using browser developer tools. • Document the before/after comparison with screenshots or recorded values.
Key Questions / Analysis	• What is the difference between the initial bundle and a lazy-loaded chunk in terms of when each is downloaded? • Why does lazy loading improve perceived performance even though the total amount of code downloaded eventually stays the same? • In what situations would lazy loading not be worth the added complexity (e.g., a very small app)?
Supplementary Problems	• Lazy load a heavy third-party component (e.g., a chart library) only when a specific page needs it. • Add a minimum-delay fallback (e.g., 300ms) to avoid a flickering loading state on fast connections. • Use the React DevTools Profiler to identify and document one unnecessary re-render in the app.
Key Skills Addressed	• Implementing route-based code splitting using React.lazy() and Suspense • Reading and interpreting the browser Network tab for chunk loading behavior • Measuring bundle size using build tooling output • Communicating a before/after performance comparison clearly
Learning Outcome	Students will be able to apply code splitting and lazy loading in React to reduce initial bundle size, and will be able to measure and explain the resulting performance impact.

Lab Session Step-by-Step

Steps

1. Record baseline metrics before optimization build the app and check bundle size:

```
npm run build
# note the output bundle size shown in the terminal
```

2. Open the Network tab in DevTools, reload the app, and note the total JS transferred and load time.
3. Convert route imports to lazy imports:

```
import { lazy, Suspense } from 'react';
const Projects = lazy(() => import('./pages/Projects'));
const Contact = lazy(() => import('./pages/Contact'));
```

4. Wrap the Routes block with Suspense and a fallback:

```
<Suspense fallback={<div>Loading page...</div>}>
  <Routes>
    <Route path="/projects" element={<Projects />} />
    <Route path="/contact" element={<Contact />} />
  </Routes>
</Suspense>
```

5. Rebuild the app and confirm separate chunk files are generated for each lazy route.
6. Reload with Network tab open, throttle to "Slow 3G", and observe the fallback UI rendering before the chunk loads.
7. Record post-optimization metrics and prepare the before/after comparison.

Tools / Technology / Resources Required

- Node.js (v18+), npm
- React 18+ (lazy, Suspense)
- Browser DevTools (Network + Performance tabs)
- Vite build output

Total Hours

- Lab Session: 2 hours
- Implementation: 1.5 hours
- Testing / Modification: 0.5 hours
- Total Engagement: 2 hours

Faculty Teaching Guide

Guidance Notes

- Tell students in advance to capture baseline screenshots before making any changes this is required evidence for the rubric and cannot be reconstructed afterward.
- Demonstrate Network throttling (Slow 3G) live so the fallback UI is actually visible on fast lab Wi-Fi the loading state may flash too quickly to notice otherwise.
- Clarify that lazy loading delays when code downloads, not how much code downloads in total over a full session.
- Walk through reading the Vite build output chunk list together once before students interpret their own.

Common Mistakes

- Not capturing baseline (before) metrics, making the comparison impossible to demonstrate.
- Wrapping individual components in Suspense unnecessarily instead of wrapping at the route level.
- Forgetting that React.lazy() only works with default exports, causing import errors.
- Testing only on fast lab Wi-Fi without throttling, so the fallback UI is never actually observed.

Troubleshooting Guide

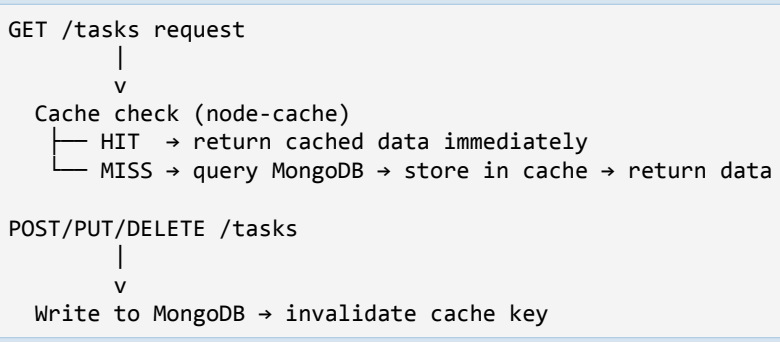
Symptom	Likely Cause	Fix
Element type is invalid error after adding lazy()	Lazy-loaded component is not a default export	Ensure the component file uses export default ComponentName
Fallback UI never appears	Chunk loads too fast on local network to observe	Enable Network throttling (Slow 3G) in DevTools to simulate a

Suspense fallback shows for the entire app, not just one route	Suspense wraps too broad a section of the component tree	Move Suspense to wrap only the Routes block, not the entire App component

Student Assignment / Post Laboratory Work	
Tasks	• Review the Theory Concepts section and the assigned Coursera module before next week. • Lazy load one additional heavy component (e.g., a chart or large list) beyond the required routes. • Use React DevTools Profiler to identify and note one unnecessary re-render in the app.
GitHub Deliverables	
Expected Repository Contents	• Same repository as Practical 6/7, updated with new commits • Lazy loading applied to at least 2 route-based components with Suspense fallback • Before/after screenshots or recorded metrics included in the README or a docs folder • At least one commit specifically for performance optimization

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
Lazy Loading Implementation (6 marks)	React.lazy() applied to at least 2 route-based components; Suspense wrapper with fallback UI present.
Code Splitting Evidence (5 marks)	Browser Network tab shows separate chunks loading per route; demonstrated live during evaluation.
Before/After Comparison (5 marks)	Screenshot or recording of bundle size and load time before and after optimization; difference explained.
Fallback UI (4 marks)	Suspense fallback renders correctly during lazy load; does not crash if chunk is slow.
Total: 20 marks	**Passing Threshold: 14 / 20 (70%)**

Practical 9: In-Memory Caching and Query Optimization	
CO/PO	CO2, CO4 / PO3, PO5
Objective	To implement server-side caching and measure its impact on API response time.
Prerequisites	• Practicals 4–7 completed working Node/Express/MongoDB backend with authentication • Basic understanding of why repeated database reads are expensive at scale
Theory Concepts (Self-Study Reference)	
Coursera Pointer	See Week 9 IBM: Node.js & MongoDB Developing Back-end Database Applications, Module 5 (optimization techniques, back-end performance, scaling approaches, load balancing overview).
Architecture / Diagram	
Diagram	
Problem Definition	
Problem Statement	• Add in-memory caching to the Task Management backend using node-cache. • Cache the response of GET /tasks (all tasks) with a reasonable TTL (e.g., 60 seconds). • Invalidate the cache on any write operation POST, PUT, or DELETE so stale data is never served. • Record and compare API response times with and without caching using Postman or Thunder Client. • Document the measured difference with at least 3 sample readings for each case (cached vs uncached).
Key Questions / Analysis	• Why must the cache be invalidated on every write operation, and what would happen to data correctness if it were not? • What is a reasonable TTL (time-to-live) for cached data in a task management context, and what trade-off does TTL length represent? • Why is in-memory caching (node-cache) not suitable for a multi-server/multi-instance deployment, even though it works fine in this lab?
Supplementary Problems	• Cache the GET /tasks/:id single-task endpoint separately from the all-tasks endpoint. • Add a cache-hit/cache-miss counter and expose it via a debug endpoint. • Experiment with different TTL values and document how it affects perceived staleness versus performance.
Key Skills Addressed	• Implementing in-memory caching using node-cache • Designing correct cache invalidation logic for write operations • Measuring and comparing API response times empirically • Reasoning about the trade-offs of caching strategies
Learning Outcome	Students will be able to implement in-memory caching for a REST API, ensure cache correctness through invalidation, and measure the performance impact using real response time data.

Lab Session Step-by-Step

Steps

1. Install node-cache:

```
npm install node-cache
```

2. Initialize a cache instance in a shared module:

```
const NodeCache = require('node-cache');
const cache = new NodeCache({ stdTTL: 60 });
module.exports = cache;
```

3. Update the GET /tasks route to check the cache first:

```
const cached = cache.get('all_tasks');
if (cached) return res.json(cached);
const tasks = await Task.find();
cache.set('all_tasks', tasks);
res.json(tasks);
```

4. Invalidate the cache inside POST, PUT, and DELETE handlers after a successful write:

```
cache.del('all_tasks');
```

5. In Postman, send GET /tasks 3 times and record response time for each (look at the "Time" field).
6. Temporarily disable caching (comment out the cache check) and repeat the same 3 requests for comparison.
7. Record both sets of readings in a simple table for the lab journal.

Tools / Technology / Resources Required

- Node.js (v18+), npm
- node-cache
- Express.js, Mongoose
- Postman or Thunder Client (for response time measurement)

Total Hours

- Lab Session: 2 hours
- Implementation: 1.5 hours
- Testing / Modification: 0.5 hours
- Total Engagement: 2 hours

Faculty Teaching Guide

Guidance Notes

- Tell students explicitly to record uncached response times BEFORE adding the cache check, or the comparison cannot be done fairly afterward.
- Clarify that node-cache is process-local memory explain briefly why this would not work correctly if the app ran across multiple server instances (sets up the later Unit 4 scaling discussion).
- Walk through a live example of forgetting cache invalidation, showing students a stale response after an update, to make the risk concrete.
- Encourage at least 3 repeated readings per condition rather than a single measurement, since response time can vary.

Common Mistakes

- Forgetting to invalidate the cache after a write, leading to stale data being served indefinitely within the TTL window.
- Caching per-request without a stable cache key, causing every request to be treated as a cache miss.
- Only taking a single response time reading instead of multiple samples, making the comparison unreliable.
- Setting a TTL so short that the cache effectively never gets hit during testing.

Troubleshooting Guide

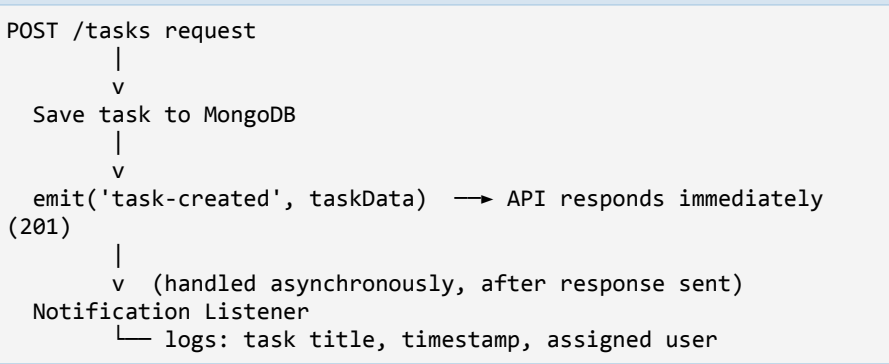
Symptom	Likely Cause	Fix
Updated task not reflected in GET response	Cache not invalidated after PUT/DELETE	Add cache.del('all_tasks') inside every write route handler
No measurable difference between cached and uncached	TTL too short or cache key changing on every request	Use a fixed, consistent cache key string and a TTL of at least 60 seconds for testing
Cache works but server restarts	node-cache is in-memory and	This is expected behavior for in-

Response time barely changes	MongoDB query was already fast (small dataset)	Add more sample task documents to the database to make the cache benefit more visible

Student Assignment / Post Laboratory Work	
Tasks	• Review the Theory Concepts section and the assigned Coursera module before next week. • Cache the single-task GET /tasks/:id endpoint separately from the all-tasks endpoint. • Add a simple cache-hit/cache-miss counter exposed via a debug endpoint.
GitHub Deliverables	
Expected Repository Contents	• Same repository as Practical 4/5, updated with new commits • node-cache implementation with correct invalidation on all write operations • Response time comparison data (cached vs uncached) included in README or docs folder • At least one commit specifically for caching implementation

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
node-cache Setup (4 marks)	node-cache installed and initialized correctly; cache instance used in route file.
Cache on GET (5 marks)	GET all tasks checks cache first; returns cached data if available; fetches from DB only on cache miss.
Cache Invalidation (5 marks)	Cache cleared correctly on POST, PUT, DELETE operations; stale data does not persist after write.
Response Time Comparison (6 marks)	Response times recorded with and without caching using Postman; measurable difference demonstrated and noted.
Total: 20 marks	**Passing Threshold: 14 / 20 (70%)**

Practical 10: Asynchronous Processing with Event-Driven Architecture	
CO/PO	CO4 / PO3, PO5
Objective	To implement asynchronous background processing using Node.js native EventEmitter without external dependencies.
Prerequisites	• Practicals 4–9 completed working Node/Express/MongoDB backend with caching • Basic understanding of synchronous vs asynchronous execution in JavaScript
Theory Concepts (Self-Study Reference)	
Coursera Pointer	No dedicated Coursera module for this topic refer to the Node.js official documentation on Events (nodejs.org/en/docs/guides/event-loop-timers-and-nexttick), supplemented by the Node.js Crash Course on the Traversy Media YouTube channel.
Architecture / Diagram	
Diagram	
Problem Definition	
Problem Statement	• Add a background notification system to the Task Management application using Node.js built-in EventEmitter. • When a new task is created via the API, emit a task-created event handled asynchronously. • The event handler must log a notification (task title, timestamp, assigned user) without blocking the main response. • The main API response must return immediately, before the event handler finishes executing. • Demonstrate the difference between synchronous and asynchronous handling by logging timestamps at both the API response point and the event handler execution point.
Key Questions / Analysis	• Why does emitting an event not block the API response, even though both run on the same Node.js process? • What would happen to API response time if the notification logic were placed directly inside the POST route instead of in an event handler? • Why is EventEmitter a reasonable choice for this scale of application, but not for a production system handling millions of events per day?
Supplementary Problems	• Add a second event, task-deleted, with its own listener that logs a different message. • Add an error event listener that catches and logs any error thrown inside the task-created handler. • Simulate a slow notification handler (e.g., setTimeout of 2 seconds) and confirm the API still responds instantly.
Key Skills Addressed	• Using Node.js built-in EventEmitter to decouple side effects from the main request flow • Designing non-blocking background processing without external message queue infrastructure • Understanding the Node.js event loop at a practical, applied level

	• Comparing synchronous and asynchronous execution using timestamp evidence
Learning Outcome	Students will be able to implement non-blocking, event-driven background processing using Node's built-in EventEmitter and explain how it keeps the main request-response cycle fast.
Lab Session Step-by-Step	

Steps	1. Create a dedicated events.js module:
	`const EventEmitter = require('events');`
	`class TaskEvents extends EventEmitter {}`
	`module.exports = new TaskEvents();`
	2. Register a listener for the task-created event, ideally in a separate listeners.js file:
	`const taskEvents = require('./events');`
	`taskEvents.on('task-created', (task) => {`
	`  console.log(`[Notification] Task "${task.title}" created at ${new Date().toISOString()}`);`
	`});`
	3. Inside the POST /tasks route, emit the event right after saving the task, then respond immediately:
	`const task = await Task.create(req.body);`
	`console.log(`[API] Response sent at ${new Date().toISOString()}`);`
	`res.status(201).json(task);`
	`taskEvents.emit('task-created', task);`
	4. Run the server and create a task via Postman; observe the console log order.
	5. Add an artificial delay inside the listener (e.g., a loop or setTimeout) to make the asynchronous behavior more visible.
	6. Confirm and record that the API response timestamp appears before the listener's completion timestamp in the console.

Tools / Technology / Resources Required	• Node.js (v18+) built-in events module • Express.js, Mongoose • Postman or Thunder Client • Visual Studio Code
Total Hours	• Lab Session: 2 hours • Implementation: 1.5 hours • Testing / Modification: 0.5 hours • Total Engagement: 2 hours
Faculty Teaching Guide	
Guidance Notes	• Clarify upfront that this practical replaces a message-queue-based approach (e.g., RabbitMQ/Bull/Redis) which is out of scope for this course EventEmitter is the deliberate, simpler substitute. • Emphasize that emit() does not wait for listeners to finish this is the core concept being tested and is easy to misunderstand as 'asynchronous magic'. • Use the console.log timestamp ordering as the main proof point during evaluation rather than relying on student explanation alone. • Connect this practical back to Practical 9's caching discussion both are examples of decoupling work from the critical request path.
Common Mistakes	• Placing taskEvents.emit() before res.status(201).json(task), which still works but obscures the ordering being demonstrated emit after the response logic is clearer pedagogically. • Writing notification logic directly inside the route handler instead of in a separate listener, defeating the purpose of the exercise. • Not adding any artificial delay or distinguishing log statements, making the synchronous vs asynchronous difference invisible in the console output. • Forgetting to register the listener (taskEvents.on(...)) before the event is ever emitted, causing the event to silently do nothing.

Troubleshooting Guide

Symptom	Likely Cause	Fix
Notification log never appears	Listener file never required/imported, so .on() was never registered	Ensure the listeners file is required once at server startup, e.g., in server.js
API response feels delayed	Listener logic accidentally placed before the response is sent, or emit() called synchronously before res.json()	Move res.status(201).json(task) to execute before taskEvents.emit(...)
Unhandled 'error' event crashes the server	EventEmitter has a special case: an unhandled 'error' event throws	Always register an error listener if emitting custom error events, or avoid using the reserved 'error' event name
Timestamps appear identical	No artificial delay added to the listener to make async timing visible	Add a short setTimeout or loop inside the listener to exaggerate the timing difference for demonstration

Student Assignment / Post Laboratory Work

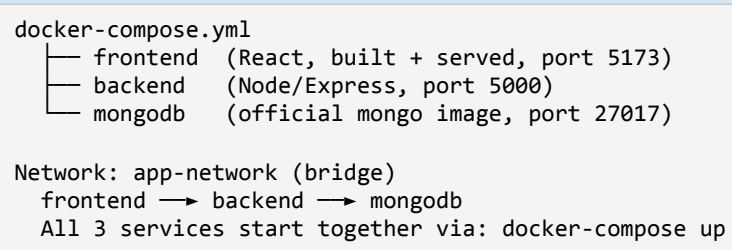
Tasks	• Review the Theory Concepts section and the Node.js Events documentation before next week. • Add a task-deleted event with its own listener and distinct log message. • Add an error event listener that catches and logs any error thrown inside the task-created handler.

GitHub Deliverables

Expected Repository Contents	• Same repository as Practical 4/5/9, updated with new commits • Working EventEmitter-based notification system with at least one custom event • Console log evidence (screenshot or recording) showing response-before-handler timestamp ordering • At least one commit specifically for event-driven async processing

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
EventEmitter Setup (4 marks)	Node.js built-in EventEmitter imported and instantiated correctly in a dedicated module.
Event Emission on Task Create (5 marks)	task-created event emitted inside POST endpoint; event carries task title, timestamp, and relevant data.
Async Handler (5 marks)	Event listener processes notification asynchronously; main API response returns before handler completes.
Timestamp Logging Evidence (6 marks)	Console shows API response timestamp before notification handler timestamp; difference demonstrated live.
Total: 20 marks	**Passing Threshold: 14 / 20 (70%)**

Practical 11: Containerization with Docker and Docker Compose	
CO/PO	CO5 / PO3, PO5
Objective	To containerize the full-stack application using Docker and orchestrate services using Docker Compose.
Prerequisites	• Practicals 1–10 completed working full-stack Task Management application • Docker Desktop (or Docker Engine) installed and verified on the lab machine
Theory Concepts (Self-Study Reference)	
Coursera Pointer	See Week 11 IBM: Introduction to Containers with Docker, Kubernetes & OpenShift (focus only on the Docker fundamentals and Docker Compose sections; Kubernetes/OpenShift content is out of scope for this course).
Architecture / Diagram	
Diagram	
Problem Definition	
Problem Statement	• Write a Dockerfile for the React frontend of the Task Management application. • Write a Dockerfile for the Node/Express backend. • Create a docker-compose.yml file that orchestrates all 3 services: frontend, backend, and MongoDB. • Configure correct port mappings and an internal Docker network so the services can communicate. • The entire application must start with a single docker-compose up command, with no manual steps in between.
Key Questions / Analysis	• Why does the backend container need to reference the MongoDB container by its service name (e.g., mongodb) rather than localhost? • What is the difference between EXPOSE in a Dockerfile and the ports mapping in docker-compose.yml? • Why is it considered bad practice to copy node_modules into a Docker image instead of running npm install inside the container?
Supplementary Problems	• Add a .dockerignore file to exclude node_modules and .env from the build context. • Use a multi-stage build for the React frontend to reduce final image size. • Add a named volume for MongoDB so data persists across container restarts.
Key Skills Addressed	• Writing Dockerfiles for both a frontend and backend Node-based application • Orchestrating multi-container applications using Docker Compose • Configuring inter-container networking and port mapping • Running and verifying a complete application stack with a single command
Learning Outcome	Students will be able to containerize a full-stack application using Docker and orchestrate multiple services together using Docker Compose, running the entire stack with one command.
Lab Session Step-by-Step	

Steps	1. Verify Docker is installed and running:
	`docker --version`
	`docker compose version`
	2. Create a Dockerfile in the backend folder:
	`FROM node:18`
	`WORKDIR /app`
	`COPY package*.json ./`
	`RUN npm install`
	`COPY . .`
	`EXPOSE 5000`
	`CMD ["node", "server.js"]`
	3. Create a Dockerfile in the frontend folder:
	`FROM node:18 AS build`
	`WORKDIR /app`
	`COPY package*.json ./`
	`RUN npm install`
	`COPY . .`
	`RUN npm run build`
	`EXPOSE 5173`
	`CMD ["npm", "run", "preview", "--", "--host"]`
	4. Create docker-compose.yml at the project root defining all 3 services with build context, ports, and depends_on.
	5. Update the backend MongoDB connection string to use the service name (e.g., mongodb://mongodb:27017/taskdb) instead of localhost.
	6. Run the entire stack:
	`docker-compose up --build`
	7. Confirm all 3 containers are running and the app is accessible end-to-end:
	`docker ps`

Tools / Technology / Resources Required	• Docker Desktop / Docker Engine • Docker Compose • Node.js (for local reference, not required at runtime once containerized) • Terminal / Command line
Total Hours	• Lab Session: 2 hours • Implementation: 1.5 hours • Testing / Modification: 0.5 hours • Total Engagement: 2 hours
Faculty Teaching Guide	
Guidance Notes	• This is an awareness-level practical, not a production Docker deep dive the goal is for students to see and run a working multi-container setup, not master Docker internals. • Pre-pull the node:18 and mongo images on lab machines before the session if internet bandwidth is limited, to avoid losing lab time to image downloads. • Demonstrate the localhost vs service-name distinction live this is the single most common point of confusion when containerizing a previously local app. • Reserve the last 15 minutes specifically for the single-command verification (docker-compose up) as a class checkpoint.
Common Mistakes	• Using localhost or 127.0.0.1 in the backend's MongoDB connection string instead of the Compose service name. • Forgetting to expose/map the correct ports, making the app inaccessible from the host browser. • Not excluding node_modules via .dockerignore, resulting in extremely slow and bloated image builds. • Running docker-compose up without --build after changing a Dockerfile, so old cached layers are used.
Troubleshooting Guide	

Symptom	Likely Cause	Fix

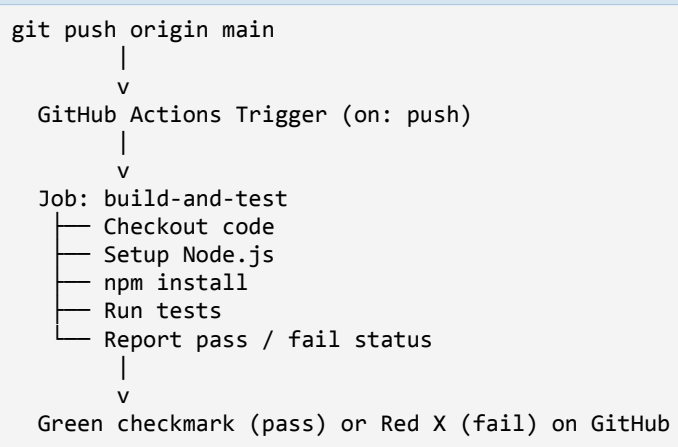
Changes to code not reflected after rebuild	Old image layers cached	Run docker-compose up --build to force a fresh image build

Student Assignment / Post Laboratory Work	
Tasks	• Review the Theory Concepts section and the assigned Coursera module before next week. • Add a .dockerignore file excluding node_modules and .env. • Add a named volume for MongoDB so task data persists across container restarts.

GitHub Deliverables	
Expected Repository Contents	• Same repository as previous practicals, updated with new commits • Dockerfiles for both frontend and backend, plus a working docker-compose.yml • README updated with instructions to run the full stack via docker-compose up • At least one commit specifically for containerization

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
Dockerfile Frontend (4 marks)	Valid Dockerfile for React app; correct base image; build and serve steps present.
Dockerfile Backend (4 marks)	Valid Dockerfile for Node/Express; correct base image; dependencies installed; app starts on container run.
Docker Compose File (6 marks)	Compose file defines all 3 services (frontend, backend, MongoDB); correct port mappings; services linked via network.
Single Command Run (6 marks)	Entire app starts with docker-compose up without manual intervention; all 3 containers running confirmed via docker ps.
Total: 20 marks	**Passing Threshold: 14 / 20 (70%)**

Practical 12: CI/CD Pipeline with GitHub Actions	
CO/PO	CO5 / PO3, PO5
Objective	To automate testing and deployment of the application using a CI/CD pipeline.
Prerequisites	• Practical 11 completed containerized full-stack application • GitHub account with the project repository already pushed • Basic understanding of YAML syntax
Theory Concepts (Self-Study Reference)	
Coursera Pointer	See Week 12 GitHub Actions Masterclass (Packt), Module 1 (workflow basics, triggers, jobs, steps) and Module 2 (CI/CD pipeline setup, automated testing on push).
Architecture / Diagram	
Diagram	
Problem Definition	
Problem Statement	• Set up a GitHub Actions workflow for the Task Management application that runs automatically on every push to the main branch. • The pipeline must install dependencies (npm install) as a defined step. • The pipeline must run at least one automated test and report a clear pass/fail status. • Demonstrate a failed pipeline caused by a deliberately broken test. • Demonstrate a fixed pipeline that passes after the test is corrected.
Key Questions / Analysis	• What is the difference between Continuous Integration and Continuous Deployment, and which one does this practical actually demonstrate? • Why does running tests automatically on every push catch problems earlier than testing manually before a release? • What would happen to the pipeline if the test file path were incorrect would the workflow fail at the same step as a genuine test failure?
Supplementary Problems	• Add a second job that runs a linter (e.g., ESLint) before the test job. • Configure the workflow to run only on pull requests targeting main, not on every direct push. • Add a status badge to the README that reflects the current pipeline status.
Key Skills Addressed	• Writing a GitHub Actions workflow YAML file • Configuring triggers, jobs, and steps for a CI pipeline • Writing at least one automated test that can run in CI • Reading and interpreting CI pipeline logs and pass/fail status
Learning Outcome	Students will be able to set up a basic CI pipeline using GitHub Actions that automatically installs dependencies and runs tests on every push, and will be able to diagnose a failing pipeline from its logs.

Lab Session Step-by-Step

Steps

1. Install a basic testing library in the backend project:

```
npm install --save-dev jest supertest
```

2. Write at least one simple test (e.g., testing the GET /tasks endpoint returns status 200):

```
test('GET /tasks returns 200', async () => {
  const res = await request(app).get('/tasks');
  expect(res.statusCode).toBe(200);
});
```

3. Create the workflow file at .github/workflows/ci.yml:

```
name: CI Pipeline
on:
  push:
    branches: [main]
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 18
      - run: npm install
      - run: npm test
```

4. Commit and push the workflow file; confirm it appears under the Actions tab on GitHub.
5. Deliberately break the test (change an expected value) and push again observe the red X / failed status.
6. Fix the test and push again observe the pipeline turn green.
7. Take screenshots of both the failed and passed pipeline runs for the lab journal.

Tools / Technology / Resources Required

- GitHub account and repository
- GitHub Actions (no separate install required)
- Jest or a similar Node.js testing library
- Git CLI, Visual Studio Code

Total Hours

- Lab Session: 2 hours
- Implementation: 1.5 hours
- Testing / Modification: 0.5 hours
- Total Engagement: 2 hours

Faculty Teaching Guide

Guidance Notes

- Confirm GitHub access and internet connectivity in the lab before this session the entire practical depends on it.
- Walk through the Actions tab UI together first so students know where to find pipeline runs and logs before debugging on their own.
- Make the deliberate test failure a required step, not optional seeing a red X and reading the failure log is the core learning moment.
- Keep the test itself trivial (e.g., a single status code check) the goal is understanding the pipeline mechanism, not testing depth.

Common Mistakes

- Placing the workflow YAML file in the wrong path (must be exactly .github/workflows/<name>.yml).
- Incorrect YAML indentation, causing the workflow to fail to even register on GitHub.
- Writing a test that always passes trivially (e.g., expect(true).toBe(true)), which defeats the purpose of the demonstration.
- Forgetting to push the workflow file itself, then wondering why nothing appears under the Actions tab.

Troubleshooting Guide

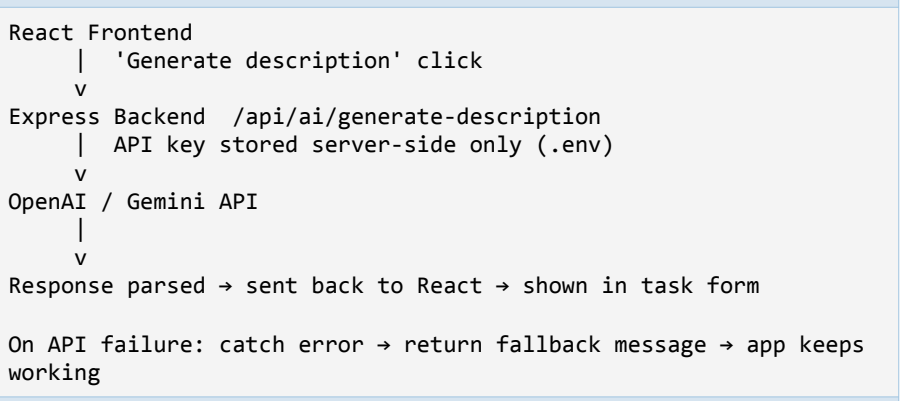
Symptom	Likely Cause	Fix

Pipeline passes even with a broken test	Test was written incorrectly and does not actually assert the intended behavior	Review the test logic to confirm it fails when the code is genuinely broken

Student Assignment / Post Laboratory Work	
Tasks	• Review the Theory Concepts section and the assigned Coursera module before next week. • Add a second automated test covering a different endpoint. • Add a status badge to the project README reflecting the current CI pipeline status.
GitHub Deliverables	
Expected Repository Contents	• Same repository as previous practicals, updated with new commits • Working .github/workflows/ci.yml with install and test steps • Screenshots of both a failed and a passed pipeline run included in README or docs folder • At least one commit specifically for CI pipeline setup

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
Workflow File (4 marks)	.github/workflows/ directory exists; YAML file is valid; trigger set to push on main branch.
Dependency Install Step (3 marks)	Pipeline runs npm install successfully as first step; logged in Actions tab.
Automated Test Step (5 marks)	At least one test written and executed in pipeline; pass/fail status visible in GitHub Actions tab.
Failed Pipeline Demo (4 marks)	Intentionally broken test causes pipeline to fail; failure reason visible in logs.
Fixed Pipeline Demo (4 marks)	Test fixed; pipeline reruns and passes; green checkmark shown in GitHub Actions.
Total: 20 marks	**Passing Threshold: 14 / 20 (70%)**

Practical 13: AI API Integration into a Web Application	
CO/PO	CO6 / PO3, PO5, PO12
Objective	To integrate an external AI service into the existing application and understand responsible AI usage.
Prerequisites	• Practical 6 completed working full-stack Task Management application • An active OpenAI API key or Gemini API key (institution-provided or free-tier)
Theory Concepts (Self-Study Reference)	
Coursera Pointer	No dedicated Coursera module refer to the YouTube video 'Build an AI Image Generator with OpenAI & Node.js' by Traversy Media for general OpenAI API integration patterns in Node.js (adapt the API call pattern to a text-based use case for this practical).
Architecture / Diagram	
Diagram	
Problem Definition	
Problem Statement	• Add an AI-powered feature to the Task Management application using the OpenAI API or Gemini API. • The feature must serve a meaningful purpose for example, auto-generating a task description, suggesting a priority level, or proposing a deadline based on the task title. • The API key must be stored and used only on the backend (.env), never exposed to the frontend. • Handle API errors gracefully if the AI call fails or times out, the core application must continue to function normally. • Display the AI-generated response meaningfully in the React UI, not as raw JSON.
Key Questions / Analysis	• Why must the AI API key be called from the backend rather than directly from the React frontend? • What does 'graceful degradation' mean in this context, and why does it matter for a feature that depends on a third-party service outside your control? • What are the privacy and governance implications of sending task data to an external AI API, and what kind of data should never be sent this way?
Supplementary Problems	• Add a loading state on the frontend while waiting for the AI response. • Add a simple rate limit (e.g., max 1 AI request per task per minute) to avoid excessive API usage. • Display a clear disclaimer in the UI that the suggestion is AI-generated and should be reviewed by the user.
Key Skills Addressed	• Calling an external AI API securely from a Node/Express backend • Designing a feature that degrades gracefully when a third-party dependency fails • Parsing and displaying AI-generated content meaningfully in a React UI

	• Reasoning about ethical, privacy, and governance implications of AI integration
Learning Outcome	Students will be able to securely integrate a third-party AI API into a full-stack application, handle failures gracefully, and articulate the responsible-AI considerations involved.
Lab Session Step-by-Step	

Steps	1. Install the OpenAI SDK (or use fetch directly for Gemini):
	`npm install openai`
	2. Store the API key in .env (never commit this file):
	`OPENAI_API_KEY=your_key_here`
	3. Create a backend route that accepts a task title and returns a generated description:
	`const OpenAI = require('openai');`
	`const client = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });`
	
	`app.post('/api/ai/generate-description', async (req, res) => {`
	`  try {`
	`  const completion = await client.chat.completions.create({`
	`  model: 'gpt-4o-mini',`
	`  messages: [{ role: 'user', content: `Write a short task description for: ${req.body.title}` }]`
	`  });`
	`  res.json({ description: completion.choices[0].message.content });`
	`  } catch (err) {`
	`  res.status(200).json({ description: '', fallback: true });`
	`  }`
	`});`
	4. On the frontend, add a "Generate description" button on the task creation form that calls this endpoint.
	5. Display the returned description as an editable suggestion the student can accept or modify.
	6. Test the failure path by temporarily using an invalid API key, and confirm the app shows a fallback message instead of crashing.

Tools / Technology / Resources Required	• Node.js (v18+), npm • OpenAI API or Gemini API (institution-provided key) • Express.js, React 18+ • Postman (for backend endpoint testing)
Total Hours	• Lab Session: 2 hours • Implementation: 1.5 hours • Testing / Modification: 0.5 hours • Total Engagement: 2 hours
Faculty Teaching Guide	
Guidance Notes	• Distribute the institution-sponsored API key/access method before the session begins do not let key provisioning eat into lab time. • Emphasize the security point strongly: the API key must never appear in any frontend code or be committed to GitHub. • Use this session to also briefly discuss Responsible AI from the syllabus what data should and should not be sent to a third-party model. • Demonstrate the graceful-degradation failure path live (using a broken key) so students see it's a designed behavior, not a bug.
Common Mistakes	• Calling the OpenAI/Gemini API directly from the React frontend, exposing the API key publicly in browser network requests. • Not handling API errors, causing the entire task creation flow to break if the AI call fails. • Committing the .env file with the real API key to GitHub. • Treating the AI response as the final answer rather than an editable suggestion, removing user control.

Troubleshooting Guide

Symptom	Likely Cause	Fix
401 Unauthorized from OpenAI/Gemini	Invalid or missing API key	Verify the key in .env is correct and the variable name matches what the code reads
API key visible in browser DevTools Network tab	AI API called directly from frontend instead of through the backend	Move the AI API call entirely to an Express route; frontend should only call your own backend endpoint
App crashes when AI call fails	No try/catch around the API call	Wrap the API call in try/catch and return a fallback response instead of letting the error propagate
Rate limit / quota exceeded error	Too many requests sent in a short time, especially across a full lab batch	Add a basic per-task rate limit and inform students in advance to avoid repeated unnecessary calls

Student Assignment / Post Laboratory Work

Tasks	• Review the Theory Concepts section and the AI integration reference video before next week. • Add a loading state on the frontend while waiting for the AI response. • Add a visible disclaimer noting the content is AI-generated and should be reviewed.

GitHub Deliverables

Expected Repository Contents	• Same repository as Practical 6, updated with new commits • Working AI-powered feature with graceful failure handling • .env excluded via .gitignore; no API key committed anywhere in history • At least one commit specifically for AI integration

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
API Integration (5 marks)	AI API (OpenAI or Gemini) called correctly from Node backend; API key stored in .env; not exposed in frontend.
Feature Meaningfulness (5 marks)	AI feature serves a real purpose in the app (auto-generate description, prioritize task, suggest deadline); not a raw prompt demo.
Graceful Degradation (5 marks)	App continues to function if AI API fails or times out; error caught and user shown a friendly fallback message.
Response Handling (5 marks)	AI response parsed correctly; displayed meaningfully in React UI; no raw JSON dumped on screen.
Total: 20 marks	**Passing Threshold: 14 / 20 (70%)**